Persona 2 · Motor de procesamiento

Tu trabajo: un comando que recibe un video y dice, por cada posición del anaquel, qué precio y SKU tiene la etiqueta, si hay hueco y cuántos frentes hay. Tú observas; P4 compara contra el sistema y calcula el dinero. Así se reparten el trabajo sin pisarse.

```
python -m motor.pipeline --video video/recorrido_realista.mp4 --planograma
datos/sistema/planograma.csv --salida salida/
```

Recuerda la regla de oro: **nunca leas `escena/` ni `video/verdad/`** desde el motor. Solo para entrenar puedes usar `video/verdad/` como anotaciones.

Tu salida: `salida/observaciones.json` (interfaz con P4)

```
{"video": "recorrido_realista.mp4", "segundos_proceso": 38,
 "posiciones": [
  {"posicion": "R1-P1", "etiqueta_visible": true, "sku_leido": "SKU-100664",
   "precio_leido": 7.09, "confianza_precio": 0.97, "lecturas_coinciden": 4,
   "hueco": false, "frentes_vistos": 3, "cuadro": 312,
   "recorte_etiqueta": "recortes/R1-P1_etiqueta.jpg", "recorte_posicion":
"recortes/R1-P1.jpg"}]}
```

Además: `salida/recorrido_anotado.mp4` con las cajas dibujadas (verde ok, rojo hueco, naranja etiqueta) para que el tablero lo reproduzca.

Módulos y orden

#	Archivo	Qué hace	Hora meta
0	Colab	Entrenar detector general de productos con un subconjunto de SKU-110K (para el clip real)	Se lanza en H0–1 y corre solo
1	`cuadros.py`	Lee el video, elige 1 cuadro nítido cada ~12 cuadros (varianza del Laplaciano)	H2
2	`marcadores.py`	Detecta ArUco y calcula la homografía imagen → centímetros de la sección	H3

#	Archivo	Qué hace	Hora meta
3	`etiquetas.py`	Encuentra etiquetas por el borde naranja, las endereza a 600 × 350 px y les asigna posición por su x en cm	H5
4	`ocr.py`	PaddleOCR sobre cada etiqueta enderezada; extrae precio y SKU; vota entre cuadros	H8
5	`huecos.py` + `frentes.py`	YOLO entrenado con la verdad de Blender (clases `etiqueta`, `producto`, `hueco`); asigna cajas a posiciones y cuenta frentes	H12
6	`anotar.py`	Video anotado y recortes	H14
7	`pipeline.py`	Une todo, mide tiempo, escribe `observaciones.json`	H10 versión 1, H16 final

Prompts para Claude Code (pega el contrato primero)

Colab, en la hora 0:

> Escribe un notebook de Google Colab que entrene YOLO nano de Ultralytics con un subconjunto de 1,500 imágenes del dataset SKU-110K (usa `SKU-110K.yaml` de Ultralytics), 30 épocas, imagen 640, una sola clase `producto`. Al final exporta `best.pt` y muéstrame mAP50 en validación. Debe poder reanudarse si Colab se desconecta.

Selección de cuadros y marcadores:

> Escribe `motor/cuadros.py` y `motor/marcadores.py`. `cuadros.py`: lee un mp4 con OpenCV, toma un cuadro de cada 12 y, dentro de cada ventana, conserva el más nítido según la varianza del Laplaciano; devuelve índice y imagen. `marcadores.py`: con la API ArUco de OpenCV ≥ 4.7 (`cv2.aruco.ArucoDetector`, diccionario `DICT_4X4_100`) detecta marcadores; decodifica sección, repisa y k con `id = seccion*20 + repisa*4 + k`; sabiendo que cada marcador mide 5 cm y su esquina superior izquierda está en x = 0, 55 o 110 cm y en la altura de su repisa (contrato), calcula la homografía de píxeles a centímetros de la sección. Incluye una función `pixel_a_cm(cuadro, x, y)`. Prueba con `video/cuadros/000001.png` y guarda en `debug/` la imagen con los marcadores marcados y una rejilla cada 10 cm.

Etiquetas y OCR:

> Escribe `motor/etiquetas.py`: encuentra rectángulos con borde naranja #FF6A00 (umbral en HSV + contornos + `approxPolyDP`), corrige perspectiva a 600 × 350 px y asigna a cada etiqueta su posición del `planograma.csv` según su x en centímetros (función de `marcadores.py`). Luego `motor/ocr.py`: usa PaddleOCR 3.x (PP-OCRv5) sin ajustar, carga el modelo una sola vez; de cada etiqueta extrae el precio con la expresión `\$?\s?(\d{1,4})[.,](\d{2})` y el SKU con `SKU-\d{6}`. Para cada posición junta lecturas de todos los cuadros y quédate con la que más se repite; reporta `lecturas_coinciden` y la confianza promedio. Si no hay lectura en ningún cuadro, `etiqueta_visible: false`. Guarda los recortes en `salida/recortes/`.

Huecos y frentes:

> Escribe `motor/preparar_yolo.py` que convierta `video/verdad/cuadro_*.json` al formato YOLO (clases 0 etiqueta, 1 producto, 2 hueco), usando 1 de cada 5 cuadros y separando 80/20 por pasada, no al azar. Dame el notebook de Colab para entrenar YOLO nano 40 épocas. Luego `motor/huecos.py`: corre el modelo en los cuadros elegidos, convierte el centro de cada caja a centímetros y asígnala a la posición cuyo rango `x_ini_cm–x_fin_cm` y repisa la contienen. Una posición es hueco si en la mayoría de sus cuadros hay caja `hueco` o no hay ninguna `producto`. `frentes_vistos` = mediana entre cuadros del número de cajas `producto` en esa posición.

Pipeline:

> Escribe `motor/pipeline.py` con la línea de comandos del contrato. Llama a los módulos en orden, mide el tiempo total, escribe `salida/observaciones.json` exactamente con el esquema dado y genera `salida/recorrido_anotado.mp4` con las cajas dibujadas. Si un paso falla en una posición, que no truene todo: marca esa posición con `confianza_precio: 0` y sigue.

Métricas que debes pasar (P4 tiene el script que las mide)

Métrica	Mínimo
Etiquetas encontradas	⩾ 95%
Precio exacto por etiqueta, tras votar	⩾ 95%
Huecos encontrados / huecos falsos	⩾ 90% / ⩽ 1 por corrida
Frentes exactos	⩾ 85% de posiciones
Tiempo para un clip de 10 s en laptop	⩽ 60 s

Cortes

- **H5:** si las etiquetas no se encuentran por color en ⩾ 90% → ubícalas por geometría: con el marcador ya sabes dónde debe estar cada etiqueta en centímetros; recorta ese rectángulo.
- **H8:** si el precio exacto < 85% → agranda 2× el recorte antes del OCR, prueba Tesseract con lista blanca `0123456789.$` y vota entre ambos; si sigue mal, pide a P1 render a 60 cm.
- **H10:** si YOLO no está listo → hueco por posición: pide a P1 un render del anaquel **vacío** con la misma cámara y compara cada posición contra ese fondo (diferencia de imagen).
- **H14:** si todo pasa sus métricas → intenta "producto en lugar equivocado": compara cada recorte de producto contra `escena/empaques/*.png` con DINOv2 o OpenCLIP (similitud coseno). Si no pasa 90% a la hora 16, se queda como futuro.
- **H16:** si tarda > 60 s → procesa menos cuadros (1 de cada 20) y guarda en caché los resultados de los clips ensayados.
- **H18:** código congelado. El clip real se corre con lo que haya; para él, usa el detector de SKU-110K del Colab.